CLASSI E OGGETTI (PARTE 1)

MODULO DI PROGRAMMAZIONE ORIENTATA AGLI OGGETTI

INFORMATICA PER LA COMUNICAZIONE DIGITALE

UNIVERSITÀ DI CAMERINO

Michele Loreti
Anno Accademico 2023/24

DALLE PROCEDURE…

Nella *programmazione imperativa* ogni programma è costituito da due tipologie di elementi base:

- le dichiarazioni delle *strutture dati*;
- le *funzioni* per elaborarle.

Il codice viene organizzato in *moduli*.
Ogni *modulo* viene pensato per svolgere un particolare compito.

DALLE PROCEDURE…

Alla base della programmazione *imperativa* abbiamo quindi una *netta separazione* tra *codice* e *strutture dati*.
L'esecuzione di un programma segue un ordine *sequenziale*: a partire da una funzione principale (tipicamente il `main`) le funzioni vengono chiamate in un ordine prestabilito che dipende dai dati.
Le strutture dati verranno inizializzate, salvate nella memoria principale (lo *heap*) e/o passate ad altre funzioni/procedure per realizzare compiti specifici.

DALLE PROCEDURE…

Per sviluppare un programma seguendo il paradigma procedurale dovremo quindi rispondere a domande del tipo:

- Di quali strutture dati ho bisogno per rappresentare le informazioni che il mio programma devo elaborare?
- Di quali funzioni ho bisogno per elaborare queste informazioni?
- In quale ordine tali funzioni devono essere elaborate?

…AGLI OGGETTI

Nella programmazione *Orientata agli Oggetti* tale separazione tra *dati* e *funzioni* viene superata.
Infatti, un programma sviluppato seguendo questo paradigma sarà costituito da un insieme di *elementi software*, denominati *oggetti*, che svolgono un duplice compito:

- rappresentare e memorizzare dell'informazione;
- fornire le funzionalità necessarie ad elaborare quell'informazione.

…AGLI OGGETTI

Nella programmazione imperativa le funzioni esistono indipendentemente dai dati. Nella programmazione orientata agli oggetti, una funzionalità esiste solo se c’è un oggetto, caricato in memoria, che la mette a disposizione.

GLI OGGETTI

Un oggetto è una componente software caratterizzata da:

- uno stato;
- un comportamento.

Lo stato di un oggetto è rappresentato dai suoi *attributi*.
Mentre il comportamento è definito mediante i *metodi*.

COSA RAPPRESENTA UN OGGETTO?

Un oggetto può essere immaginato come una sorta di *mattoncino* che mette a disposizione alcune funzionalità.
Il programma verrà poi realizzato come un insieme di oggetti.
Infatti, come una *costruzione complessa* si ottiene da piccoli elementi, così nella programmazione OO un programma nasce dalla composizione di oggetti.

LA PROGRAMMAZIONE ORIENTATA AGLI OGGETTI

L’obiettivo della programmazione OO è quello di *progettare* (o *utilizzare*) gli oggetti adeguati a risolvere il nostro problema.
Ogni oggetto, a sua volta, può essere internamento costruito attraverso la composizione di altri oggetti, fornendo specifiche funzionalità, permettono di costruirne una più complessa.
L’obiettivo è quello di costruire una *rete di oggetti* che, grazie alla loro *interazione*, realizzeranno la computazione desiderata

QUALI DOMANDE?

Le domande che dovremo porci sono:

- Di quali oggetti ho bisogno per realizzare il mio programma?
- Quali funzionalità devono avere?
- Come li posso comporre tra loro?

MA PERCHÉ?

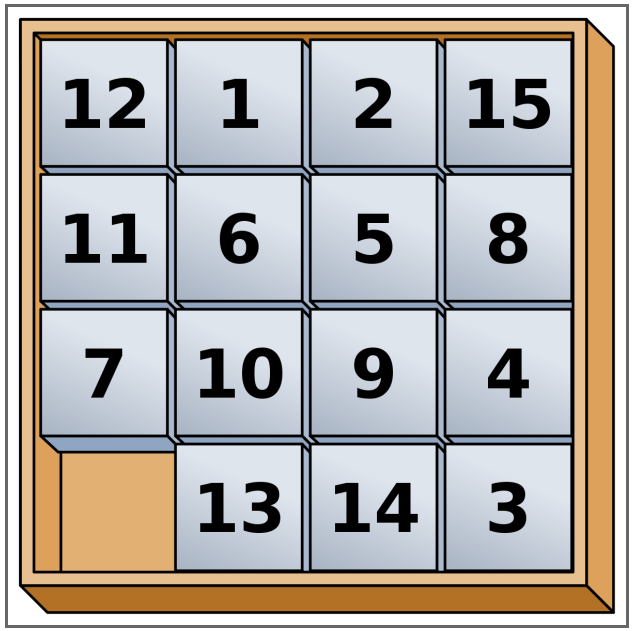
Ma perché è necessario utilizzare un simile paradigma? Ci sono dei vantaggi evidenti?
Il principale vantaggio nell'uso della programmazione orientata agli oggetti risiede in tre principali ragioni:

- *riusabilità*;
- *manutenibilità*;
- *estendibilità del codice*.

Per ottenere tali vantaggi, però, è necessario sviluppare buon codice.
In questo corso cercheremo di capire insieme cosa vuol dire scrivere *buon codice*.

UN ESEMPIO

Consideriamo, ad esempio, il gioco del 15 che utilizzeremo come esempio per richiamare ed introdurre costrutti Java.



UN ESEMPIO

Partiamo da alcune domande:

- Quale è lo stato del gioco del 15?
- Quale è il suo comportamento?

Rispondere a queste domande è il primo passo verso la costruzione di un programma OO.

LA CLASSE

Per classe si intende il *prototipo* per un insieme di oggetti.
La classe rappresenta quindi un insieme di oggetti.
Se vogliamo implementare il Gioco del 15 dobbiamo, per prima cosa, definire la classe che ne rappresenta gli oggetti.

LE CLASSI IN JAVA

In Java una classe ha la seguente sintassi:

```
public class NomeClasse {

}
```

CLASSI, PACCHETTI E FILE

In Java le classi fanno parte di un *pacchetto* (*package*).
Un package viene usato per raggruppare classi che condividono finalità.
Ad esempio, se guardiamo le librerie standard in Java abbiamo:

- `java.math`
- `java.net`
- `java.io`

CLASSI, PACCHETTI E FILE

Il nome di un pacchetto dovrebbe individuare in modo *univoco* chi ha sviluppato quel codice.
Per questa ragione, la struttura dei pacchetti segue lo stesso schema delle URL, ma in modo invertito:

- `org.apache.commons.math3`

DICHIARAZIONE DEL PACCHETTO

La dichiarazione del pacchetto deve essere inserita all'inizio del file dove si trova la classe:

```
package nome.pacchetto;

public class NomeClasse {

}
```

CLASSI, PACCHETTI E FILE

Per fare riferimento ad una classe `ClasseA` dichiarata nel pacchetto `a.b.c` viene univocamente identificata come `a.b.c.ClasseA`.
Se vogliamo usare questa classe in un pacchetto diverso da `a.b.c`, sarà conveniente importare la classe, o il pacchetto all'inizio del file, dopo la dichiarazione del package:

```
import a.b.c.ClasseA;
```

CLASSI, PACCHETTI E FILE

Il nome del paccetto viene utilizzato anche per ricercare la classe.
Per questa ragione, la classe `a.b.c.ClasseA` dichiarata nel pacchetto `a.b.c` deve essere dichiarata nel file `ClasseA.java` che si trova all'interno del path `a/b/c/` nella cartella dei sorgenti.
Inoltre, per poter essere trovata, la classe deve essere dichiarata `public`.

I CAMPI

La dichiarazione dei campi di un oggetto è simile a quella delle variabili.
Oltre al tipo, al nome del campo e all'inizializzazione, i campi possono avere dei *modificicatori*:

- di *visibilità*: `public`, `protected`, `private`;
- di *accesso*: `static`, `final`.

I MODIFICATORI DI VISIBILITÀ

Permettono di limitare l'accesso ai campi/metodi messi a disposizione di un oggetto:

- `public`: accesso consentito a tutti;
- `protected`: accesso consentito solo all'interno dello stesso package (o in relazione di ereditarietà);
- `private`: accesso consentito solo dalla stessa classe.

I MODIFICATORI DI ACCESSO

I modificatori di accesso vengono utilizzati per restringere le modalità di uso di un campo/metodo.

- `final`: questo modificatore quando applicato ad un campo indica che il suo valore non potrà mai cambiare;
- `static`: indica un campo/metodo associao alla classe, se abinato al modificatore `final` indica delle costanti.

ESEMPIO: IL GIOCO DEL 15

Quali sono i campi della classe del gioco del 15?
Una possibilità è quella di utilizzare i campi per rappresentare:

- il contenuto delle celle;
- la coordinata della cella libera.

```
public class PuzzleBoard {

    private int[][] cells;
    private int emptyX;
    private int emptyY;
```

ESEMPIO: IL GIOCO DEL 15

L’array (bi-dimensionale) che mantiene le informazioni sulle celle ha la seguente struttura:

```
[0,0] [1,0] [2,0] [3,0]
[0,1] [1,1] [2,1] [3,1]
[0,2] [1,2] [2,2] [3,2]
[0,3] [1,3] [2,3] [3,3]
```

I METODI

Un metodo rappresenta una funzionalità associata all'oggetto.
Un metodo:

- riceve dei parametri;
- può cambiare lo stato dell'oggetto;
- restituisce un valore (opzionale).

La dichiarazione del metodo ha la seguente struttura:

```
seq_modificatori tipo nomeMetodo(seq_parametri)
    blocco
```

ESEMPIO: IL GIOCO DEL 15

Quali sono i metodi che ci aspettiamo di trovare nel nostro oggetto che rappresenta il gioco del 15?
Tutti i metodi che *mimano* le operazioni che facciamo sullo schema del 15!
Ad esempio:

- spostare un tassello;
- ottenere il valore in una certa posizione;
- verificare se siamo in una posizione finale.

ESEMPIO: IL GIOCO DEL 15

```
public boolean moveUp( ) {
    if (emptyY<3) {
        cells[emptyX][emptyY] = cells[emptyX][emptyY+1];
        emptyY++;
        cells[emptyX][emptyY] = 0;
        return true;
    }
    return false;
}
```

I COSTRUTTORI

Dopo aver definito una classe, dobbiamo definirne il *costruttore*.
Questo è un metodo particolare, senza tipo di ritorno e con nome uguale a quello della classe, che permette descrivere come *costruire* una *istanza* della classe.
Se una classe non contiene nessun costruttore, Java ne assegna uno senza parametri.

ESEMPIO: IL GIOCO DEL 15

Nel nostro caso il costruttore dovrà:

- impostare il valori nelle celle (ossia i numeri tra 0 e 15);
- posizionare la cella vuota (posizione `(0,3)`).

ESEMPIO: IL GIOCO DEL 15

```java
public PuzzleBoard() {
    this.cells = new int[4][4];
    reset();
}
private void reset() {
    this.emptyX = 3;
    this.emptyY = 3;
    int counter = 1;
    for(int i=0; i < 4; i++) {
        for(int j=0; j < 4; j++) {
            this.cells[i][j] = (counter++)%16;
        }
    }
}
```

COME VALUTIAMO QUESTA SOLUZIONE?

La soluzione proposta fino ad ora è sicuramente adeguata.
Con uno sguardo più attento, però, possiamo notare come ci sia del codice ripetuto: i metodi `moveUp`, `moveDown`, `moveLeft` e `moveRight` differiscono in pochi punti.
Come possiamo *semplificare* la soluzione? Quali sono i punti critici?

PROVIAMO A SEMPLIFICARE

Possiamo notare come nei metodi ci sia un parametro *nascosco*.
Questo parametro è, di fatto, la *direzione* del movimento! Possiamo notare che questo è un concetto che, se pur presente nel gioco, non è stato *rappresentato nella soluzione proposta*.
Per rappresentare la direzione possiamo utilizzare delle enumerazioni.

LE ENUMERAZIONI

In Java è possibile definire delle *enumerazioni*.
Queste rappresentano un insieme finito di valori a cui possiamo, opzionalmente, associare delle funzionalità.
Nel nostro esempio del Gioco del 15, ad esempio, possiamo utilizzarle per definire le possibili direzioni dello spostamento.

ESEMPIO...

```
public enum SlidingDirection {
    UP,
    DOWN,
    LEFT,
    RIGHT;
}
```

IL COSTRUTTO `switch-case`

Le enumerazioni vengono spesso utilizzate con il costrutto `switch-case` che permette di selezionare differenti computazioni in funzione del risultato della valutazione di un'espressione:

```
switch (expr) {
    case  expr1: blocco1
    case  expr2: blocco2
    ...
    default blocco3
}
```

IL COSTRUTTO `switch-case`

Possiamo usare questo costrutto per *semplificare* il nostro codice:

```
public boolean move(SlidingDirection dir) {
    switch (dir) {
        case UP: return moveUp();
        case DOWN: return moveDown();
        case LEFT: return moveLeft();
        case RIGHT: return moveRight();
        default: return false;
    }
}
```

LA COMPOSIZIONE DI OGGETTI

Abbiamo visto come definire un oggetto.
Per realizzare un programma quello che dobbiamo fare è mettere insieme più oggetti che *interagiscono* tra loro per *realizzare* la computazione.
Se svogliamo bene questa attività di *composizione* il nostro codice diventerà molto semplice!

CODICE RIPETUTO

Uno dei vantaggi derivanti dall'uso della composizione degli oggetti è la riduzione del *codice ripetuto*.
Se osserviamo i metodi `moveUp`, `moveDown`, `moveLeft` e `moveRight` ci possiamo rendere conto che hanno molto codice in comune.
Perché?

CODICE RIPETUTO

Uno dei vantaggi derivanti dall'uso della composizione degli oggetti è la riduzione del *codice ripetuto*.
Se osserviamo i metodi `moveUp`, `moveDown`, `moveLeft` e `moveRight` ci possiamo rendere conto che hanno molto codice in comune.
Perché? Perché in questi metodi noi facciamo due cose:

- determiniamo la posizione del tassello che viene spostato;
- eseguiamo l'operazione di spostamento.

CODICE RIPETUTO

Per questa ragione possiamo introdurre una specifica classe che può essere utilizzata per gestire le *posizioni* nello schema del gioco.
Come al solito dobbiamo partire da due domande:

- quale è lo stato di una posizione?
- quali sono le sue funzionalità?

LA CLASSE `Position`

```java
public class Position {

    private final int row;

    private final int column;

    private final int size;

    public Position(int row, int column, int size) {
        this.row = row;
        this.column = column;
        this.size = size;
    }
```

LA CLASSE `Position`

```
public Position next(SlidingDirection dir) {
  switch (dir) {
    case UP:
      if (row < size-1)
        return new Position(row+1,column,size);
    case DOWN:
      if (row > 0)
        return new Position(row-1,column, size);
    case LEFT:
      if (column > 0)
        return new Position(row, column-1, size);
    case RIGHT:
      if (column < size-1)
        return new Position(row, column+1, size);
  }
  return null;
}
```

L’ESPRESSION

A partire da Java 19, è possibile utilizare un’epsressione di tipo switch.
La sintassi è simile a quella del comando:

```
switch (<expr>) {
        (case <label> -> <expr>;)*
        default -> expr;
}
```

Questo rientra in una serie di interventi su Java che mirano a ridurre la *verbosità* del linguaggio.
Torneremo sull’argomento in seguito.

LA CLASSE `Position`

```
public Position next(SlidingDirection dir) {
  return switch (dir) {
    case UP:    (row < size-1?new Position(row+1,column,size):null);
    case DOWN:  (row > 0?new Position(row-1,column,size):null);
    case LEFT:  (column > 0?new Position(row,column-1,size):null);
    case RIGHT: (column < size-1?new Position(row,column-1,size):null);
  }
}
```

THE SIMPLER, THE BETTER!

Usando la classe `Position`, il codice dello spostamento diventa molto più semplice:

```
public boolean move(SlidingDirection dir) {
    Position movingCell = freeCell.next(dir);
    if (movingCell == null) {
        return false;
    }
    set(freeCell, get(movingCell));
    freeCell = movingCell;
    return true;
}
```

ESERCIZIO: COME POSSIAMO MESCOLARE LE CASELLE?

Ancora una volta possiamo utilizzare i singoli oggetti per svolgere i compiti elementari.
Mescolare le caselle vuol dire eseguire una sequenza di mosse *random*.
Per mischiare le caselle dobbiamo ripetere un certo numero di volte:

- determinare le mosse disponibili;
- sceglierne una ed eseguirla.

ESERCIZIO: COME POSSIAMO MESCOLARE LE CASELLE?

Per determinare le mosse disponibili possiamo arrichire la classe `Position` aggiungendo un modo opportuno:

```
public SlidingDirection[] enabledMoves() {
...
}
```

ESERCIZIO: COME POSSIAMO MESCOLARE LE CASELLE?

Per determinare le mosse disponibili possiamo arrichire la classe `Position` aggiungendo un modo opportuno:

```
public void shuffle(Random random, int movements) {
 for(int i=0; i < movements; i++) {
  SlidingDirection[] enabledMovement = enabledMoves();
  if (enabledMovement.length > 0) {
   int idx = random.nextInt(enabledMovement.length);
   move(enabledMovement[idx]);
  }
 }
}
public SlidingDirection[] enabledMoves() {
 return freeCell.enabledMoves();
}
```

SCHEMA RISOLTO…

Come possiamo fare a determinare se lo schema è risolto?
Una possibile soluzione è quella di andare a *controllare* se le schede sono nella posizione corretta.
Questo però è computazionalmente inefficiente!

SCHEMA RISOLTO…

Abbiamo due approcci per risolvere questo problema:

- contare il *numero di celle fuori posto*;
- calcolare il *grado di disordine* dello schema.

Entrambi gli approcci sono, di fatto, una applicazione della *memoization*.
Questa è una tattica per *salvare* i risultati intermedi per un loro riuso successivo.

LA SOLUZIONE!

Entrambi gli approcci proposti si basano sulla possibilità di determinare la posizione *corretta* di una *cella*.
Data una cella c, come possiamo definire la posizione *corretta* $pos(c) = (x_c, y_c)$ della cella?
Basta un semplice conto:

$$c = (x_c + y_c \cdot 4) + 1$$

$$c - 1 = x_c + y_c \cdot 4$$

$$x_c = (c-1)\%4$$

$$y_c = (c-1)/42$$

NUMERO DI CELLE FUORI POSTO…

Per mantenere un contatore del numero di celle fuori posto dobbiamo inanzitutto aggiungere un contatore allo stato dell'oggetto:

```
private int numberOfCorrectCells;
```

NUMERO DI CELLE FUORI POSTO...

Quando facciamo uno spostamento, è sufficiente andare ad aggiornare tale contatore:

```
public boolean move(SlidingDirection dir) {
    Position movingCell = freeCell.next(dir);
    if (movingCell == null) {
        return false;
    }
    int value = getValueAt(movingCell);
    if (movingCell.isCorrect(value)) numberOfCorrectCells--;
    if (freeCell.isCorrect(value)) numberOfCorrectCells++;
    set(freeCell, get(movingCell));
    freeCell = movingCell;
    return true;
}
```

DISTANZA DI MANHATTAN

Date due coppie $p_1 = (x_1,y_1)$ e $p_2 = (x_2,y_2)$ di interi, indichiamo con *distanza di Manhattan* la funzione δ_M definita come:

$$\delta_M(p_1,p_2) = |x_1-x_2| + |y_1-y_2|$$

Questa *distanza* misura il numero *minimo* di mosse necessarie per spostare un tassello dalla posizione p_1 alla posizione p_2!

GRADO DI CONFUSIONE…

Data una configurazione C, il suo grado di confusione χ può essere definito come segue:

$$\chi(C) = \sum_i \sum_j \delta_M(pos(C[i,j]),(i,j))$$

GRADO DI CONFUSIONE...

Seguendo un approccio simile a quello che abbiamo visto precedentemente, possiamo aggiornare il grado di confusione, che all'inizio è 0 ad ogni mossa:

```java
private int shuffleDegree = 0;

public boolean move(SlidingDirection dir) {
    Position movingCell = freeCell.next(dir);
    if (movingCell == null) {
        return false;
    }
    shuffleDegree -= getShuffleDegree(movingCell);
    set(freeCell, get(movingCell));
    shuffleDegree += getShuffleDegree(freeCell);
    freeCell = movingCell;
    return true;
}
```

SCHEMA RISOLTO…

Per verificare la se lo schema è risolto o meno sarà sufficiente verificare il valore degli attributi!
Nel caso del contatore delle celle fuori posto:

```
public boolean isSolved() {
    return (this.numberOfCorrectCells==15);
}
```

Nel caso del grado di disordine:

```
public boolean isSolved() {
    return (this.shuffleDegree==0);
}
```

ALCUNE CONSIDERAZIONI…

Siamo partiti dalla definizione di una classe, `PuzzleBoard`, per implementare lo schema del gioco del 15.
A partire da una soluzione *funzionante* abbiamo prodotto attraverso passi successivi una il cui codice è *migliore*:

- meno codice replicato;
- metodi più semplici e *focalizzati* (fanno una cosa sola).

TO BE CONTINUED...

Speaker notes